

WEEK 6 · DAY 1

LLMOps Lifecycle & AI Evaluation

Prompt Versioning, AIP Evals, and Observability — Hardening What Weeks 2–5 Built

1-Hour Lecture Session — Concepts & Architecture

Rajesh Pasham

Program Context

How today's session fits the wider 8-week delivery

Session Type	Concepts and architecture lecture
Audience	India and Europe batches together — this session is common to all four batches
This Week's Focus	Production AIP, LLMOps & Governance — hardening the Copilots, chatbots, and agent workflows built in Weeks 2–5
This Week's Deliverable (Day 3)	A Production Readiness Package: an evaluation suite, an observability dashboard, and a release plan

Where Today Fits

From working prototypes to systems a production operation can trust

Weeks 2 through 5 built things that work — a Claims Copilot that answers questions, chatbots and functions that write back to the Ontology, and agents that plan and act on their own. None of that was tested the way a production system needs to be. Week 6 is where you stop asking “does it work in the demo?” and start asking “how do we know it still works, every day, for every policyholder, and how would we know the moment it doesn't?”

Not new theory in isolation — it is the LLM-powered systems you've already built, now held to a production bar.

Today's four topics — the LLMOps lifecycle, prompt versioning, model evaluation, and observability — are the first half of that bar.

Tomorrow's session covers the second half: drift monitoring, cost management, release strategy, and governance. Concretely: your Batch 1 ClaimsCopilot, Batch 2 UnderwritingCopilot, Batch 3 FraudTriageCopilot, and Batch 4 ServicingCopilot — along with the FraudRiskAssessment, RiskScoreRefresh, ComplianceReadinessCheck, and CoverageRecommendationAssessment functions and chatbots you built in Week 5 — are exactly the systems today's four topics apply to. Keep yours in mind as you go; you'll use it as the worked example throughout.

Today's Roadmap

Four topic groups, building toward Wednesday's Production Readiness Package

1

The LLMOps Lifecycle

How operating an LLM-powered system differs from classical MLOps, and why continuous evaluation replaces one-time testing

2

Prompt Versioning

Why a prompt is a first-class artifact that needs the same discipline as code, not a string buried in a function

3

Model Evaluation with AIP Evals

Evaluation Suites, Target Functions, Evaluation Functions, Test Cases, and Metrics — applied to a function you've already built

4

AI Observability

Monitoring, debugging, and analysis for LLM-powered functions and chatbots already running in your Ontology

By the End of Today

The vocabulary Wednesday's self-paced build assumes

- ✓ **Explain the LLMOps lifecycle and how it differs from classical MLOps**

Why prompts, non-determinism, and continuous evaluation change the operating model

- ✓ **Explain why prompts require version control discipline**

And how prompt versioning connects to the system/task prompts built into Week 5's chatbots and functions

- ✓ **Describe the core concepts behind AIP Evals**

Evaluation Suite, Target Function, Evaluation Function, Test Cases, and Metrics

By the End of Today

The vocabulary Wednesday's self-paced build assumes (cont'd)

- ✓ **Apply those concepts to a function or chatbot you've already built**

Framing an evaluation plan for one of your program's existing use cases

- ✓ **Describe what Monitoring, Debugging, and Analysis each cover in Foundry Observability**

And how they apply to the LLM-powered systems already running in your Ontology

- ✓ **Preview Day 2 and the Week 6 deliverable**

So Wednesday's self-paced build starts from a shared vocabulary, not a blank page

TOPIC 1

The LLMOps Lifecycle

How operating an LLM-powered system differs from classical MLOps

From MLOps to LLMOps

A familiar discipline, applied to a very different kind of model

Classical MLOps assumes a deterministic, numeric model

A fraud-scoring model trained on structured data gives the same score for the same input, every time. Your own RiskScoreRefresh function (Batch 2) has exactly this kind of piece inside it — the riskScoreStale check is a plain deterministic calculation, which is why it was never built as a Use LLM block.

An LLM-powered function does not give that guarantee

The same prompt, sent to the same model, can return a differently worded — or differently reasoned — response on two separate calls. Feed your Batch 1 ClaimsCopilot the same claim narrative twice and its summary wording can legitimately differ both times — that is the non-determinism this week's evaluation discipline exists to manage.

From MLOps to LLMOps

A familiar discipline, applied to a very different kind of model (cont'd)

The unit of operation changes too

Classical MLOps versions a trained model artifact. LLMOps versions a model choice, a prompt, and the tools a function or chatbot can call. Your Batch 3 Compliance Chatbot is a visible example: its model choice, its system prompt, and its access to the ComplianceReadinessCheck and SubmitComplianceReview tools are three separate things that each need to be tracked, not one.

What the LLMOps Lifecycle Actually Covers

A continuous loop, not a one-time deployment checklist

Build

The AIP Logic functions and AIP Chatbot Studio chatbots you've been building since Week 2 — prompts, tool access, and model selection, authored and iterated. Your Batch 1 FraudRiskAssessment function and Fraud Detection Chatbot are a concrete instance of this stage: a Use LLM fraud-judgment block wired to an OpenInvestigation tool.

Evaluate

Testing behavior against defined test cases before and after any change — covered in depth later today through AIP Evals. The Evaluation Suite requirement on your Batch 2 RiskScoreRefresh function (3+ Test Cases before it counts as done) is this stage in practice, not theory.

What the LLMOps Lifecycle Actually Covers

A continuous loop, not a one-time deployment checklist (cont'd)

Observe & Operate

Monitoring behavior once a function or chatbot is in live use, catching regressions and failures quickly. Opening your Batch 3 Compliance Chatbot's own Monitoring tab to confirm it actually called `ComplianceReadinessCheck` is a small, real example of this — today's Observability topic, with drift and cost covered tomorrow.

Govern & Release

Controlled rollout of changes, audit trails, and responsible-AI guardrails. The confirmation-required setting on your Batch 4 `EscalateForCoverageRecommendation` Action — a person must approve it before it writes to the Ontology — is already a governance control you built, not a future one. Day 2's focus, previewed briefly at the end of today.

Classical MLOps vs. LLMOps

The same operational discipline, applied to a fundamentally different kind of artifact

Dimension	Classical MLOps	LLMOps
What's versioned	A trained model artifact	A model choice, a prompt, and the tools it can call
Determinism	Same input → same output, reliably	Same input can produce different output — non-determinism is expected
Testing cadence	Validate once against a held-out dataset before release	Continuous evaluation — after every prompt change, model update, or tool change
What “drift” means	The input data distribution shifts over time	Output quality or behavior can shift even with no visible input change
Primary new artifact	Training data and model weights	The prompt — now reviewed, versioned, and tested like code

Classical MLOps vs. LLMOps — In Your Own Systems

The same five dimensions, grounded in functions and chatbots you've already built

What's versioned

Your Batch 4 CoverageRecommendationAssessment function bundles a model choice, a recommendation-judgment prompt, and access to the EscalateForCoverageRecommendation tool — that bundle, not a single model file, is what now has to be versioned together.

Determinism

Run your Batch 1 ClaimsCopilot on the same claim narrative twice and expect the wording to vary slightly — that variability is normal for an LLM-powered function in a way it would never be for a classical scoring model.

Classical MLOps vs. LLMOps — In Your Own Systems

The same five dimensions, grounded in functions and chatbots you've already built (cont'd)

Testing cadence

A Test Case for FraudRiskAssessment (Batch 1) — claimAmount = 48,000, isHighValueClaim = true, expected recommendedFraudAction = OpenInvestigation — needs to be re-run every time the function's prompt or model changes, not just once before go-live.

What “drift” means

Your Batch 2 RiskScoreRefresh function's judgment can shift after a platform-level model update even though riskScoreStale's inputs and logic never changed — that is LLM drift, distinct from the input-distribution drift classical MLOps watches for.

Classical MLOps vs. LLMOps — In Your Own Systems

The same five dimensions, grounded in functions and chatbots you've already built (cont'd)

Primary new artifact

The system prompt inside your Batch 3 Compliance Chatbot — the text instructing it to weigh evidenceType coverage and missingEvidence before calling ComplianceReadinessCheck — is now a reviewed, versioned artifact in its own right, the same way a training script would be in classical MLOps.

Why Continuous Evaluation Replaces One-Time Testing

A test suite run once at launch answers a question that keeps changing

The model underneath can change

A platform-level model update can shift how a prompt is interpreted, even when nothing in your function or chatbot changed at all — including the Use LLM risk-judgment block inside your Batch 2 RiskScoreRefresh function, whose deterministic riskScoreStale calculation would stay identical while its judgment output quietly shifts.

The prompt itself changes over time

A system or task prompt refined after user feedback is a new artifact — it needs the same evaluation the original version had, not an assumption that “it still works.” The instructions inside your Batch 3 Compliance Chatbot's system prompt are exactly this kind of artifact once someone edits them.

Why Continuous Evaluation Replaces One-Time Testing

A test suite run once at launch answers a question that keeps changing (cont'd)

Non-determinism means one passing run isn't proof

Because the same input can produce different outputs across calls, a single successful test run is weaker evidence than it would be for a deterministic model. This is why the Evaluation Suite built for your Batch 1 FraudRiskAssessment function is required to carry 3+ Test Cases and get run repeatedly — one pass proves far less than it would for a classical model.

TOPIC 2

Prompt Versioning

Why a prompt needs the same discipline as code

The Prompt Is a First-Class Artifact

Not a string buried inside a function — a piece of the system that deserves its own history

Recall Week 5's system and task prompts

Every chatbot built in AIP Chatbot Studio, and every AIP Logic function using a Use LLM block, is shaped as much by its prompt as by the model behind it. Your Batch 2 Underwriting Chatbot's system prompt — describing its persona, scope, and the rule to call UnderwritingDecisionReasoning before InvokeRiskAssessmentChatbot — is doing as much work as the model choice underneath it.

A prompt change is a behavior change

Editing a task prompt's wording, tone, or instructions changes what the function or chatbot does. If someone loosened the instruction inside your Batch 3 Fraud Detection Chatbot's system prompt that it "never states a fraud read without having called [its] tool," the chatbot could start guessing — the same category of change a code edit would be, and it deserves the same scrutiny.

The Prompt Is a First-Class Artifact

Not a string buried inside a function — a piece of the system that deserves its own history (cont'd)

Without versioning, a regression is invisible until a user notices

If nobody recorded what the prompt said last week, there is no way to confirm whether this week's change caused a complaint — or to revert to the version that worked. A silent edit to your Batch 4 Customer Support Chatbot's escalation rule, with no record of the prior wording, is exactly this risk.

What Good Prompt Versioning Looks Like

The same habits you already apply to code, applied to prompt text

Every meaningful edit is a new, labeled version

Not a silent overwrite — a record of what changed, when, and by whom, the same way a commit records a code change. This is the same discipline the lab walkthroughs apply when they ask you to Publish FraudRiskAssessment's (Batch 1) final version only after a deliberate change, rather than overwriting it mid-edit.

A version is tied to the evaluation results it produced

Each prompt version should be traceable to the AIP Evals run that validated it — so “this is the version we approved” is a verifiable claim, not a memory. The 3+ Test Cases required for CoverageRecommendationAssessment (Batch 4) are what that specific published version was actually checked against.

What Good Prompt Versioning Looks Like

The same habits you already apply to code, applied to prompt text (cont'd)

Rollback is a real, exercised option

If a new prompt version underperforms once live, reverting to the last known-good version should be fast and unambiguous. If a revised judgment prompt inside your Batch 2 RiskScoreRefresh function started mis-scoring stale risk reads, a recorded prior version is what makes reverting a two-minute fix instead of a reconstruction project.

Prompt Versioning in Practice: A Chatbot Example

Tracing one prompt change through your Batch 4 Customer Support Chatbot, from idea to validated release

The Customer Support Chatbot's task prompt is edited to handle a new retention-offer scenario

1

The change — tightening when it should hand off to the Policy Recommendation Chatbot via `EscalateForCoverageRecommendation` — is written up as a new version, not applied silently over the previous wording.

The new version is run through its Evaluation Suite before anything else happens

2

The same test cases used to validate the prior version, plus new ones covering the retention-offer handoff, including a case like POL268's Renewal REN031.

Metrics from the new version are compared against the prior version's

3

Not just “does it pass” — whether accuracy and tone metrics hold steady or improve on the existing test cases, and whether the handoff to `CoverageRecommendationAssessment` still fires at the right moments.

Prompt Versioning in Practice: A Chatbot Example

Tracing one prompt change through your Batch 4 Customer Support Chatbot, from idea to validated release (cont'd)

Only once Metrics are acceptable does the new version replace the old one in production

4

The prior version stays recorded and retrievable, so a regression discovered later can be rolled back to a known point rather than reconstructed from memory.

TOPIC 3

Model Evaluation with AIP Evals

Testing LLM-powered functions and chatbots, given non-determinism

Why AIP Evals Exists

A purpose-built testing environment for the kind of non-determinism Topic 1 described

A conventional unit test assumes one correct output

An LLM-powered function may produce several differently worded but equally valid outputs — two runs of your Batch 1 ClaimsCopilot against the same claim narrative can both be correct while using different phrasing. AIP Evals is built to score that kind of result, not just an exact string match.

It evaluates AIP Logic functions, AIP Chatbot Studio chatbots, and code-authored functions

Any of the function types you've built across this program can be the subject of an evaluation — the same mechanism applies to your Batch 1 FraudRiskAssessment function and to your Batch 1 Fraud Detection Chatbot that calls it.

Why AIP Evals Exists

A purpose-built testing environment for the kind of non-determinism Topic 1 described (cont'd)

It integrates with AI FDE for conversational creation of evaluations

An Evaluation Suite can be scaffolded through a conversational interface rather than built field by field — the same kind of time saving as using Add as test case straight from a Preview run of ComplianceReadinessCheck (Batch 3), rather than typing each Test Case by hand.

The Five Core Concepts of AIP Evals

Each one answers a specific question about how a function is tested

Evaluation Suite

The overarching collection that ties everything together — it combines a set of Test Cases, one or more Target Functions, and the Evaluation Function(s) used to judge them. The Evaluation Suite built for your Batch 1 FraudRiskAssessment function is exactly this: its 3+ Test Cases, FraudRiskAssessment as the Target Function, and an Evaluation Function comparing its fraudRiskTier output to what was expected.

Target Function

The function actually being evaluated — an AIP Logic function, a chatbot function, or a code-authored function. Your Batch 2 RiskScoreRefresh function is a Target Function; so, separately, could be the Risk Assessment Chatbot (Batch 2) that calls it, since a single Suite can evaluate more than one Target Function at once.

The Five Core Concepts of AIP Evals (cont'd)

Evaluation Function, Test Cases, and Metrics complete the picture

Evaluation Function

The method used to compare a Target Function's actual output against the expected output defined in a test case. For ComplianceReadinessCheck (Batch 3), that means checking whether the function's regulatorySignOffRequired and complianceReadinessTier outputs match what the test case expected — the scoring logic behind the Suite.

Test Cases

Defined sets of inputs and expected outputs, passed into the Evaluation Function(s) each time a Suite runs. A concrete Test Case for FraudRiskAssessment (Batch 1): input claimAmount = 48,000 and isHighValueClaim = true, expected output recommendedFraudAction = OpenInvestigation.

The Five Core Concepts of AIP Evals (cont'd)

Metrics tie the cycle together

Metrics

The quantifiable, per-test-case results a Suite run produces — enabling both aggregate scoring across all test cases and comparison across multiple runs. Running the CoverageRecommendationAssessment (Batch 4) Suite five times and watching whether its pass rate on the EscalateForCoverageRecommendation test cases stays stable is exactly this kind of comparison.

Together, they form a repeatable evaluation cycle

Define Test Cases once, run the Suite against any Target Function whenever a prompt, model, or tool changes, and compare Metrics across runs rather than trusting any single run in isolation — the cycle your Batch 1 FraudRiskAssessment Suite is built to run every time that function is touched.

How an Evaluation Suite Run Works

The path from a defined test case to a trustworthy metric



A Suite run passes each Test Case's input into the Target Function, hands the Target Function's actual output and the Test Case's expected output to the Evaluation Function, and records the result as a Metric. Walked through concretely: a Test Case with `claimAmount = 48,000` and `isHighValueClaim = true` goes into your Batch 1 FraudRiskAssessment Target Function; its actual `recommendedFraudAction` output is compared by the Evaluation Function against the expected `OpenInvestigation`; the match or mismatch becomes a Metric. Because of LLM non-determinism, the value of AIP Evals comes from running this cycle repeatedly and comparing Metrics across runs — not from a single pass.

Worked Example: Evaluating FraudRiskAssessment (Batch 1)

Applying the five concepts to a function this program has already built

Target Function

Your Batch 1 FraudRiskAssessment function — it takes a claim's claimAmount and isHighValueClaim and recommends a fraudRiskTier and recommendedFraudAction, such as OpenInvestigation.

Test Cases

Representative claims with a known, agreed-upon expected recommendation each — for example, claimAmount = 48,000 and isHighValueClaim = true, expecting recommendedFraudAction = OpenInvestigation, plus edge cases near the 25,000 High-Value threshold.

Evaluation Function

A comparison method judging whether FraudRiskAssessment's recommendedFraudAction and fraudRiskTier match what each Test Case expected — in substance, not necessarily in exact wording.

Worked Example: Evaluating FraudRiskAssessment (cont'd)

Reading the Metrics and deciding whether the function is ready

Metrics, run one

The Suite runs once against all Test Cases, producing a per-test-case pass/fail — did recommendedFraudAction match expected? — plus an aggregate pass rate across the whole set.

Metrics, compared across runs

The same Suite is run again — because of non-determinism, a single run's aggregate score is not proof of reliability. Comparing pass rates across several runs of the 48,000-claim Test Case is what tells you whether FraudRiskAssessment is consistently correct or only correct by chance.

The decision this supports

Only once Metrics are both acceptable in aggregate and stable across repeated runs does this become evidence FraudRiskAssessment is ready for a production release — the same bar any prompt version or model update should be held to going forward.

TOPIC 4

AI Observability

Seeing how a function or chatbot is actually behaving, once it is live

Why Observability Is a Separate Concern from Evaluation

Evaluation asks “is this correct?” — observability asks “what is actually happening?”

Evaluation is proactive, run against known test cases

AIP Evals tells you how FraudRiskAssessment (Batch 1) performs on the claimAmount = 48,000 Test Case you defined in advance — it cannot tell you what real policyholders are actually asking the Claims Chatbot, or how the function behaves on a claim pattern nobody anticipated.

Observability is live, running against real traffic

Foundry Observability covers what is happening right now, across real executions — opening your Batch 3 Compliance Chatbot's Monitoring tab to see its actual conversations with real users, not test cases, is this in practice.

Why Observability Is a Separate Concern from Evaluation

Evaluation asks “is this correct?” — observability asks “what is actually happening?” (cont'd)

A production-ready system needs both

Passing Metrics from your Batch 2 RiskScoreRefresh Evaluation Suite and a clean observability picture for the live Risk Assessment Chatbot answer different questions — neither one substitutes for the other.

Foundry Observability: Monitoring

Keeping a continuous eye on data health and system behavior

Data Health and rule-based monitoring

Conditions defined against your Functions, Actions, and AIP Logic executions, checked continuously. A rule against your Batch 1 FraudRiskAssessment function — alert if its failure rate crosses a set threshold — is checked continuously rather than only when someone happens to look.

Alerts reach the team where it already works

Rule violations can notify through PagerDuty, Slack, webhooks, or Foundry's own notifications. A spike in failed calls to InvokeFraudDetectionChatbot (Batch 1) reaching your team's Slack channel the moment it happens is the practical form of this.

Thirty days of performance data

Enough history to spot a trend — a slow increase in failure rate, or P95 execution duration creeping upward for your Batch 2 RiskScoreRefresh function — not just a single bad moment.

Foundry Observability: Debugging

Understanding exactly what happened on one execution, after the fact

Workflow Lineage and execution history

Seven days of execution history, letting you trace a specific run of a Function, Action, or AIP Logic chain back through what it actually did — for example, following one SubmitComplianceReview (Batch 3) submission back through ComplianceReadinessCheck to the Fraud Case that triggered it.

Log search across executions

Searching logs across many executions at once, rather than opening one run at a time — searching every InvokePolicyRecommendationChatbot (Batch 4) call from the past week for a shared error signature, when you're looking for a pattern rather than a single incident.

Tracing

The ability to drill into any single operation within an execution to see its duration, its inputs, its outputs, and any errors. Tracing into one failed run of FraudRiskAssessment (Batch 1) to see the exact claimAmount it received and where in the chain it errored is the level of detail that turns “it failed” into “here is exactly where, and why.”

Foundry Observability: Analysis

Taking observability data beyond the built-in dashboards

Export logs, metrics, and traces to a streaming dataset

Observability data isn't locked inside a fixed view — a month of your Batch 3 Compliance Chatbot's traces can be exported and analyzed for a slow shift in how often it recommends escalation, a question no single built-in view answers directly.

Confirmed tracked metrics

Success and failure counts, P95 execution duration, and execution counts across resources — tracked for your Batch 2 RiskScoreRefresh function and the Risk Assessment Chatbot calling it, the same way as for any other Function, Action, or AIP Logic chain.

Foundry Observability: Analysis

Taking observability data beyond the built-in dashboards (cont'd)

Where LLM-specific tracking goes further

Token usage, cost per call, and output-quality drift are LLMOps practices your team should establish deliberately, not platform features to assume exist out of the box. This mirrors a well-known pattern in production LLM deployments more broadly: teams commonly track token volume and per-call cost in their own dashboard, the same way they'd track any other cost that can grow quietly with traffic. Day 2 returns to cost management directly.

Tying Observability Back to a Function You've Built

What Monitoring, Debugging, and Analysis each answer for a live chatbot or function

Monitoring answers: is anything wrong right now?

A rule-based alert on your Batch 2 Risk Assessment Chatbot's failure rate crossing a threshold, routed to Slack the moment it happens, rather than discovered a week later from a complaint.

Debugging answers: what exactly happened on this one run?

Tracing into a single failed execution of CoverageRecommendationAssessment (Batch 4) to see the exact proposedPremium input, the exact recommendationTier output, and where in the chain it went wrong.

Analysis answers: what's the pattern across many runs?

Exporting a month of your Batch 3 Compliance Chatbot's traces to look for a slow shift in how often it recommends escalation — the kind of question no single dashboard view is built to answer on its own.

Looking Ahead: Day 2 and the Week 6 Deliverable

Today built the vocabulary; tomorrow and Wednesday put it to work

Day 2 — Production, Cost & Governance

Drift monitoring, cost management, release strategy, capacity planning, AI governance and responsible AI, and audit and compliance — the second half of what production-readiness requires.

Day 3 — Self-paced build

You will build an evaluation suite, an observability dashboard, and a release plan for one of your program's existing functions or chatbots — today's four topics, applied hands-on.

Day 4 and 5 — Batch practicals and review

Working sessions and review of the Production Readiness Package each batch produces, closing out Week 6.

Common Pitfalls

Patterns worth avoiding before Wednesday's self-paced build



Treating a working demo as evidence the system is production-ready

Your Batch 1 ClaimsCopilot answering well in ten test conversations has not been evaluated — LLM non-determinism means it can fail on the eleventh in a way none of your ten runs revealed.



Editing a system prompt directly in production with no history

A prompt is as central to behavior as code. Editing your Batch 4 Customer Support Chatbot's escalation rule without a version history makes it impossible to know what changed, or to roll back when a change goes wrong.



Running an evaluation once, before launch, and never again

An Evaluation Suite run once at launch for RiskScoreRefresh (Batch 2) tells you nothing about a later model update, a prompt edit six weeks on, or a slow drift in output quality — evaluation is continuous, not a one-time gate.

Common Pitfalls

Patterns worth avoiding before Wednesday's self-paced build (cont'd)



Treating Metrics from a single run as conclusive

LLM non-determinism means one run's Metrics for ComplianceReadinessCheck (Batch 3) can look fine by chance — AIP Evals' value is comparing variance across multiple runs before trusting a result.



Assuming no errors in the logs means the system is behaving correctly

CoverageRecommendationAssessment (Batch 4) can execute successfully, with no error, and still return the wrong recommendationTier — observability and evaluation answer two different questions, and you need both.



Bolting on evaluation and observability at the end of a project

Both are easiest to set up while you still remember what FraudRiskAssessment (Batch 1) was supposed to do — treat them as part of this week's build, not an afterthought before go-live.

Recap & What's Next

What We Covered Today

- Why Weeks 2–5's working Copilots, chatbots, and agents still need to be hardened for production
- The LLMOps lifecycle, and how it differs from classical MLOps — prompts as a first-class artifact, non-determinism, continuous evaluation
- Why prompt versioning matters, tied back to the system/task prompts in Week 5's chatbots and functions
- AIP Evals: Evaluation Suite, Target Function, Evaluation Function, Test Cases, and Metrics
- Applying AIP Evals conceptually to a function or chatbot you've already built
- Foundry Observability's Monitoring, Debugging, and Analysis capabilities
- A first look at Day 2 and the Week 6 Production Readiness Package

Day 2 — Drift Monitoring, Cost Management, Release Strategy & Governance